CSEN 177: Operating Systems
Lab 2: Programming in C and Use of System Calls

Objectives

1. To develop sample C programs
2. To develop programs with two or more processes using fork(), exit(), wait(), and exec() system calls

Guidelines

For COEN 177L, you must develop a good knowledge of C in a Linux development environment. You are highly encouraged to use command-line tools in developing, compiling, and running your programs.

Please pay attention to your coding style and good programming practices; if your program is not worth documenting, it will not be worth running.

Skills in developing multi-processing and multi-threading applications are required to create parallelism. A process is defined as a program in execution. Multiple processes can execute in the same program, with each process owning its copy of the program within its own address space and executing it independently of the other processes.

A fork() system call is used in Linux to create a child process. The fork() takes no arguments, returns 0 for the child process, and returns the process ID (PID) for the parent process. After a new child process is created, the parent and the child processes execute the next instruction following the fork() system call. Therefore, the returned value of the fork() is used to distinguish between the parent and child processes.

- If fork() returns a negative value, the system call has failed to execute in the kernel.
    - If this is happening on ECC machines, save your work, then enter `kill -9 -1` to kill all processes under your username. If that doesn't work, power cycle the PC entirely (or switch machines).
- If fork() returns a zero, the program is the child process
- If fork() returns a positive value, the program is the parent, and the value is the process ID of its new child.

Include the following libraries for defining the process pid_t type and using fork().

```
#include <sys/types.h>
#include <unistd.h>
```

Note that the child process inherits an exact copy of the parent's address space, and both have separate address spaces. There are two special types of processes:

Zombie Process: A process is a zombie process if it is no longer active but still has an entry in its parent's process table. To avoid this, it is recommended that a child process use the exit() system call at the end of its execution. The exit status of the child process is then passed to the parent process, which removes its entry.

Orphan Process: A process is an orphan if its parent process no longer exists. This is usually because the parent process is either finished or terminated before the child process is completed. To avoid this, it is recommended that a parent process use the wait() system call, which waits for exactly one child process to complete.

C Program with two or more processes

Demonstrate each of the following steps to the TA to get a grade on this part of the lab assignment.

Step 1. [10%] Write the following C program in a Linux environment using vi.

```
/* C program to demonstrate the use of fork()*/
#include <stdio.h>      /* printf, stderr */
#include <sys/types.h>  /* pid_t */
#include <unistd.h>     /* fork */
```

```c
#include <stdlib.h>     /* atoi */
#include <errno.h>      /* errno */
/* main function */
int main() {
   pid_t  pid;
   int i, n = 3000; // n is a delay in microseconds inserted in parent and child iterations
   printf("\n Before forking.\n");
   pid = fork();
   if (pid < 0) {
      fprintf(stderr, "can't fork, error %d\n", errno);
          exit(0);
   }
   if (pid){
      // Parent process: pid is > 0
      for (i=0;i<10;i++) {
         printf("\t \t \t I am the parent Process displaying iteration %d \n",i);
         usleep(n);
      }
      wait();
   }
   else{
      // Child process: pid = 0
      for (i=0;i<10;i++) {
         printf("I am the child process displaying iteration %d\n",i);
         usleep(n);
      }
   }
   return 0;
}
```

Step 2. [10%] Compile the program using gcc by typing gcc YourProgram.c –o ExecutableName. When it compiles without errors or warnings, run the program by typing ./ExecutableName and take note of your observations.

Step 3. [10%] Rewrite the program in Step 1 so that the delay is determined by the user entered as a command line argument. In this case, the main function should be replaced by

```c
/* main function with command-line arguments to pass */
int main(int argc, char *argv[]) {
```

The value of n then is taken as

```c
n = atoi(argv[1]); // n microseconds is taken as command-line argument
```

Compile and run the program by typing `./ExecutableName 3000`.

Consider adding the following code to ensure the presence of an argument (otherwise it will segfault):

```c
if (argc != 2){
        printf ("Usage: %s <delay> \n",argv[0]);
        exit(0);
 }
```

Step 4. [10%] Note that the delay in the loop depends on the command line argument you give; here, the delay is 3000 microseconds. What happens when you change the delay to 500 and 5000?

Step 5. [10%] Rewrite the program in Step 3 to become 4 processes (including the parent) so that each process iterates with a different delay taken as a command-line argument. In this case, you will have 4 command-line arguments. When your program compiles without errors or warnings, upload the source file to Camino.

Step 6. [15%] Write a C program that becomes 5 processes (including the parent) so that each process iterates with a different delay taken as a command-line argument. In this case, you will have 5 command-line arguments. When your program compiles without errors or warnings, upload the source file to Camino.

Changing the context of the process

Processes are often created to run separate programs. This uses the exec() system call to replace the process's execution image with an entirely separate program. Although the process loses its code space, data space, stack, and heap, it retains its process ID, parent and child processes, and open file descriptors. Six versions of exec() exist, but the simplest and most widely used is execlp():

- execlp(char *filename, char *arg0, char arg1,..... , char *argn, (char *) 0);
- e.g. execlp("sort", "sort", "-n", "foo", 0);
    - Note that arg0 is (generally) a copy of the filename!

Step 7. [20%] Rewrite the program in Step 1, so that the child process runs the ls command only, and the parent process just waits until the child process terminates before exiting. You may use this code snippet:

```
if(pid == 0)
  {
     execlp("/bin/ls", "ls", 0);
  }
 else
  {
     wait(0);
     printf("Child has completed the task! \n");
     exit(0);
  }
```

Requirements to complete the lab

1. Demo to the TA the correct execution of your C programs. Consider creating a Makefile and a shell script to automate the compilation and running of all your programs as an easy way to demo to the TA.
2. Upload your observations to Camino.
3. Upload your source code for all your programs as .c file(s) to Camino.

Please start each program/text file with a descriptive block that includes the following information at a minimum:

```
# Name: <your name>
# Date: <date of lab>
# Title: Lab2 – Step <step#> - <task>
# Description: This program computes <complete description here>
```